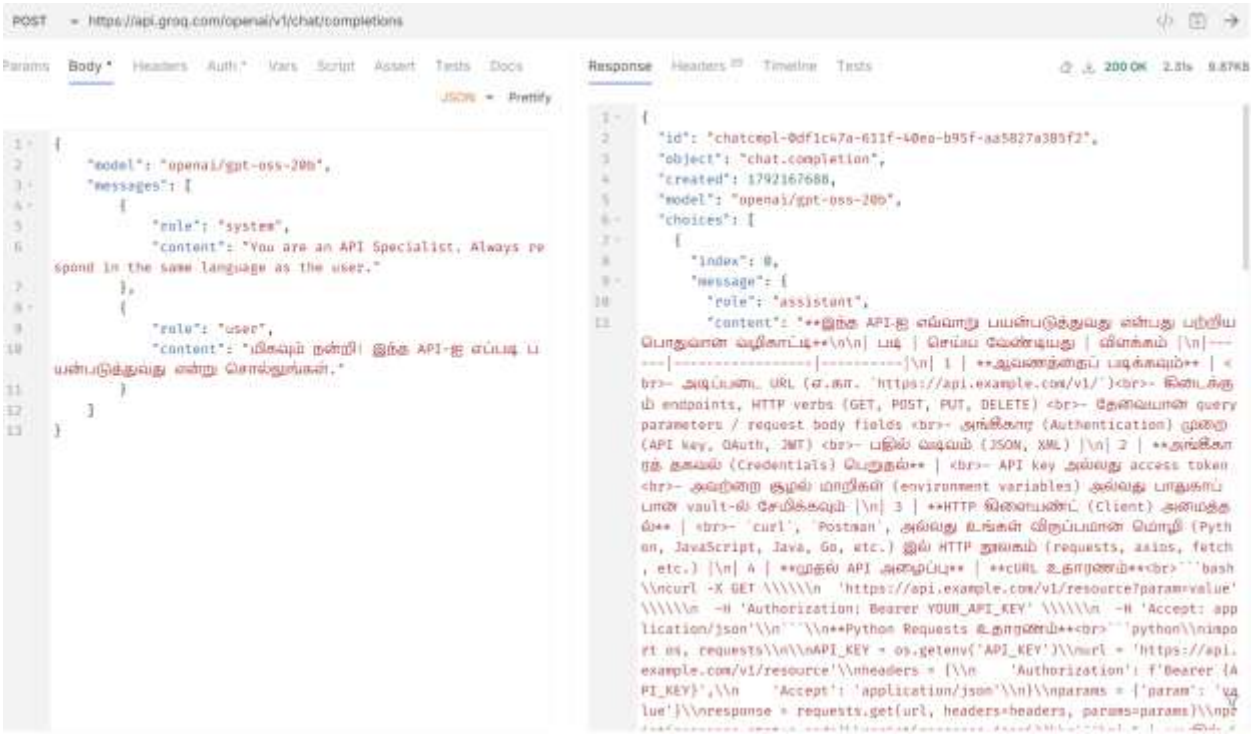
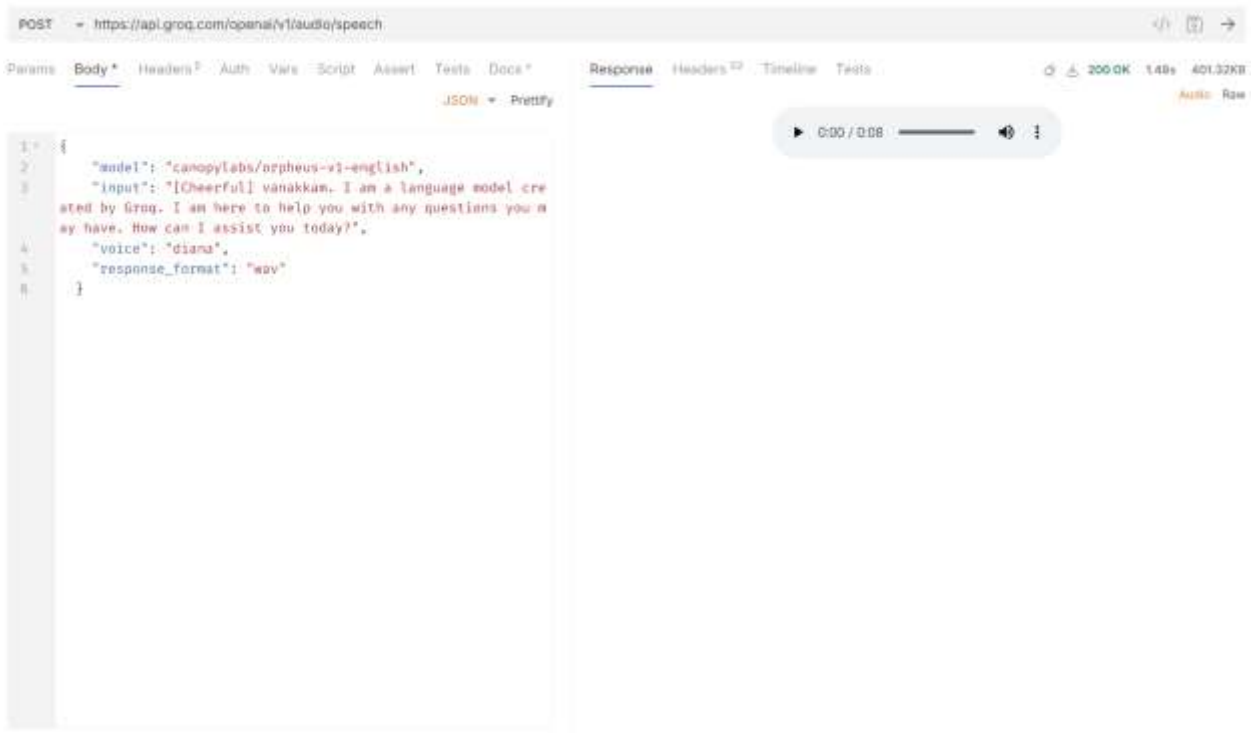
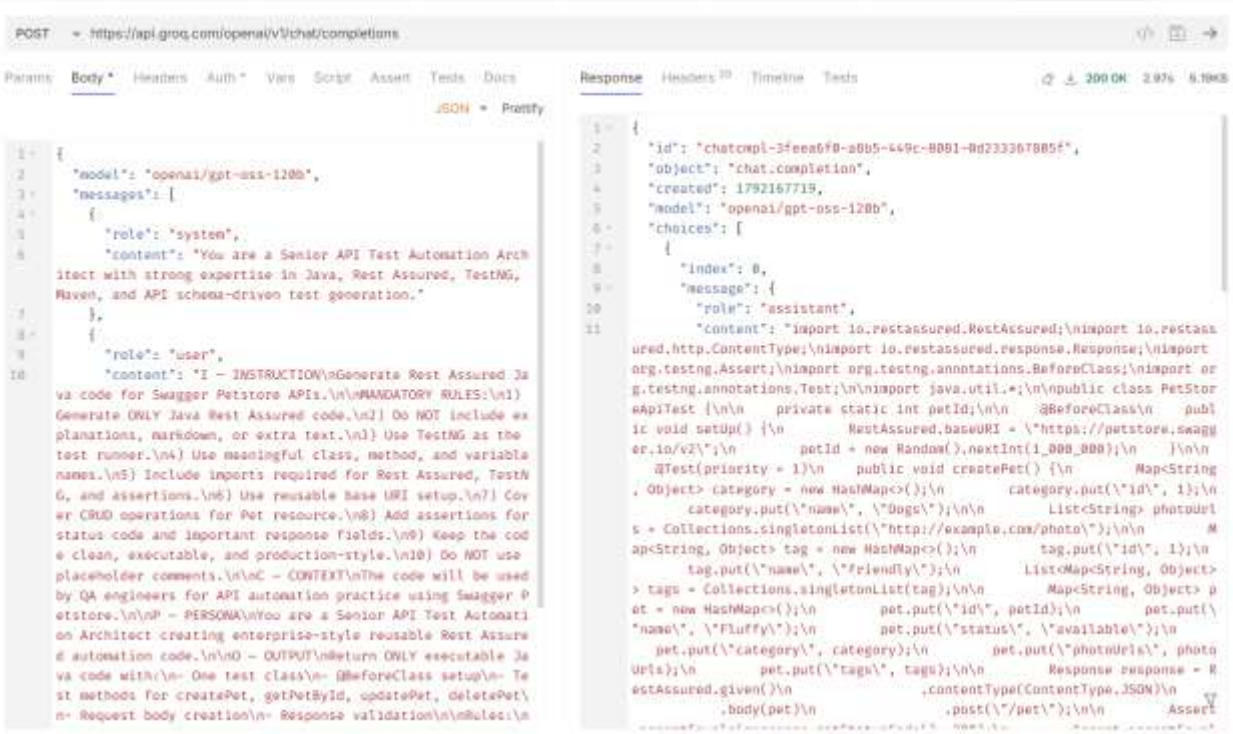
1.


POST
https://api.groq.com/openai/v1/chat/completions
Params
Body *
Headers
Auth *
Vars
Script
Assert
Tests
Docs
JSON
Prettify
{
"model": "openai/gpt-oss-20b",
"messages": [
{
"role": "system",
"content": "You are an API Specialist. Always respond in the same language as the user."
},
{
"role": "user",
"content": "மிகவும் நன்றி! இந்த API-ஐ எப்படி பயன்படுத்துவது என்று சொல்லுங்கள்."
}
]
}
Response
Headers
Timeline
Tests
200 OK
2.31s
9.87KB
{
"id": "chatcmpl-0df1c47a-611f-40ee-b95f-aa5827a385f2",
"object": "chat.completion",
"created": 1792167688,
"model": "openai/gpt-oss-20b",
"choices": [
{
"index": 0,
"message": {
"role": "assistant",
"content": "**இந்த API-ஐ எப்படி பயன்படுத்துவது என்பது பற்றிய பொதுவான வழிகாட்டி**\n\n| படி | செய்ய வேண்டியது | விளக்கம் |\n|---|---|---|\n| 1 | **ஆவணத்தைப் படிக்கவும்** |
- அடிப்படை URL (எ.கா. `https://api.example.com/v1/`)
- கிடைக்கும் endpoints, HTTP verbs (GET, POST, PUT, DELETE)
- தேவையான query parameters / request body fields
- அங்கீகார (Authentication) முறை (API key, OAuth, JWT)
- பதில் வடிவம் (JSON, XML) |\n| 2 | **அங்கீகார தகவல் (Credentials) பெறுதல்** |
- API key அல்லது access token
- அவற்றை சூழல் மாறிகள் (environment variables) அல்லது பாதுகாப்பான vault-ல் சேமிக்கவும் |\n| 3 | **HTTP கிளையண்ட் (Client) அமைத்தல்** |
- `curl`, `Postman`, அல்லது உங்கள் விருப்பமான மொழி (Python, JavaScript, Java, Go, etc.) இல் HTTP நூலகம் (requests, axios, fetch, etc.) |\n| 4 | **முதல் API அழைப்பு** | **cURL உதாரணம்**
```bash\\ncurl -X GET \\\\\\n `https://api.example.com/v1/resource?param=value' \\\\\\n -H 'Authorization: Bearer YOUR_API_KEY' \\\\\\n -H 'Accept: application/json'\\n```\\n**Python Requests உதாரணம்**
```python\\nimport os, requests\\n\\nAPI_KEY = os.getenv('API_KEY')\\nurl = 'https://api.example.com/v1/resource'\\nheaders = {\\n 'Authorization': f'Bearer {API_KEY}',\\n 'Accept': 'application/json'\\n}\\nparams = {'param': 'value'}\\nresponse = requests.get(url, headers=headers, params=params)\\npr

2.


POST
https://api.groq.com/openai/v1/audio/speech
Params
Body *
Headers
Auth
Vars
Script
Assert
Tests
Docs *
JSON
Prettify
{
"model": "canopylabs/orpheus-v1-english",
"input": "[Cheerful] vanakkam. I am a language model created by Groq. I am here to help you with any questions you may have. How can I assist you today?",
"voice": "diana",
"response_format": "wav"
}
Response
Headers
Timeline
Tests
200 OK
1.48s
401.32KB
Raw
0:00 / 0:08

3.


POST
https://api.groq.com/openai/v1/chat/completions
Params
Body *
Headers
Auth *
Vars
Script
Assert
Tests
Docs
JSON
Prettify
{
"model": "openai/gpt-oss-120b",
"messages": [
{
"role": "system",
"content": "You are a Senior API Test Automation Architect with strong expertise in Java, Rest Assured, TestNG, Maven, and API schema-driven test generation."
},
{
"role": "user",
"content": "I — INSTRUCTION\nGenerate Rest Assured Java code for Swagger Petstore APIs.\n\nMANDATORY RULES:\n1) Generate ONLY Java Rest Assured code.\n2) Do NOT include explanations, markdown, or extra text.\n3) Use TestNG as the test runner.\n4) Use meaningful class, method, and variable names.\n5) Include imports required for Rest Assured, TestNG, and assertions.\n6) Use reusable base URI setup.\n7) Cover CRUD operations for Pet resource.\n8) Add assertions for status code and important response fields.\n9) Keep the code clean, executable, and production-style.\n10) Do NOT use placeholder comments.\n\nC — CONTEXT\nThe code will be used by QA engineers for API automation practice using Swagger Petstore.\n\nP — PERSONA\nYou are a Senior API Test Automation Architect creating enterprise-style reusable Rest Assured automation code.\n\nO — OUTPUT\nReturn ONLY executable Java code with:\n- One test class\n- @BeforeClass setup\n- Test methods for createPet, getPetById, updatePet, deletePet\n- Request body creation\n- Response validation\n\nRules:\n
Response
Headers
Timeline
Tests
200 OK
2.97s
6.19KB
{
"id": "chatcmpl-3feea6f0-a0b5-449c-8081-0d233367885f",
"object": "chat.completion",
"created": 1792167719,
"model": "openai/gpt-oss-120b",
"choices": [
{
"index": 0,
"message": {
"role": "assistant",
"content": "import io.restassured.RestAssured;\nimport io.restassured.http.ContentType;\nimport io.restassured.response.Response;\nimport org.testng.Assert;\nimport org.testng.annotations.BeforeClass;\nimport org.testng.annotations.Test;\n\nimport java.util.*;\n\npublic class PetStoreApiTest {\n\n private static int petId;\n\n @BeforeClass\n public void setUp() {\n RestAssured.baseURI = \"https://petstore.swagger.io/v2\";\n petId = new Random().nextInt(1_000_000);\n }\n\n @Test(priority = 1)\n public void createPet() {\n Map<String, Object> category = new HashMap<>();\n category.put(\"id\", 1);\n category.put(\"name\", \"Dogs\");\n\n List<String> photoUrls = Collections.singletonList(\"http://example.com/photo\");\n\n Map<String, Object> tag = new HashMap<>();\n tag.put(\"id\", 1);\n tag.put(\"name\", \"Friendly\");\n List<Map<String, Object>> tags = Collections.singletonList(tag);\n\n Map<String, Object> pet = new HashMap<>();\n pet.put(\"id\", petId);\n pet.put(\"name\", \"Fluffy\");\n pet.put(\"status\", \"available\");\n pet.put(\"category\", category);\n pet.put(\"photoUrls\", photoUrls);\n pet.put(\"tags\", tags);\n\n Response response = RestAssured.given()\n .contentType(ContentType.JSON)\n .body(pet)\n .post(\"/pet\");\n\n Assert